

Deadlock vs. Starvation

FOSD – Unit 4: Deadlock Modeling and System-level Handling

Prashant Jadav

Department of Computer Engineering
DEPSTAR – CHARUSAT

Learning Objectives

By the end of this lecture, students should be able to:

- Define **deadlock** and **starvation**.
- Distinguish their causes and process behavior.
- Identify deadlock and starvation from execution scenarios.
- Explain how resource allocation and scheduling policies can cause starvation.
- Explain why deadlock recovery can itself create starvation.
- Describe techniques for reducing starvation.

Deadlock: Quick Review

Deadlock

A set of processes is permanently blocked because each process is waiting for a resource or event that can only be supplied by another process in the same set.

$$P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_1 \rightarrow P_1$$

- Processes involved in the cycle cannot proceed.
- With single-instance resource types, a cycle in the resource/wait-for representation indicates deadlock.
- The problem is associated with a cyclic waiting dependency.

Starvation: Definition

Starvation

Starvation, also called **indefinite blocking**, occurs when a process waits for CPU time or a required resource for an indefinitely long period because other processes repeatedly receive service first.

Simple example

Suppose a low-priority process P_1 is waiting for the CPU while a continuous stream of higher-priority processes keeps arriving.

$$P_1 : \text{Ready} \rightarrow \text{Ready} \rightarrow \text{Ready} \rightarrow \dots$$

P_1 may remain ready but never receive CPU time.

Deadlock vs. Starvation: Core Difference

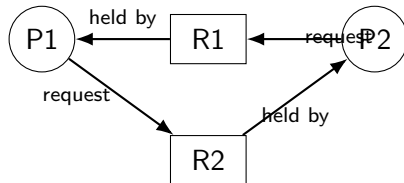
Aspect	Deadlock	Starvation
Meaning	Processes wait in a blocking dependency	A process waits indefinitely for service
Affected processes	Usually a group forming a dependency cycle	May be one process
System progress	Affected processes make no progress	Other processes may continue progressing
Typical cause	Cyclic resource dependency	Unfair scheduling/allocation
Example	P1 waits for P2 and P2 waits for P1	Low-priority P1 repeatedly bypassed
Typical remedy	Prevention, avoidance, detection/recovery	Fairness, aging, bounded waiting

Deadlock Example

Consider two processes and two resources.

Current state

- P1 holds R1.
- P2 holds R2.
- P1 waits for R2.
- P2 waits for R1.



Result

P1 and P2 are permanently blocked unless the system performs recovery or another external action breaks the cycle.

Starvation Example: Priority Scheduling

Suppose lower numerical values represent higher priority.

Process	Priority
P1	10 (low)
P2	2 (high)
P3	1 (high)
P4	3 (high)

If higher-priority processes continuously arrive:

$$P_3 \rightarrow P_2 \rightarrow P_4 \rightarrow P_3 \rightarrow P_2 \rightarrow \dots$$

P1 may remain ready but repeatedly lose CPU selection.

System progresses, but P1 may wait indefinitely

Priority scheduling is a common source of starvation when waiting processes are not given increasing priority.

Deadlock vs. Starvation: Process State

Deadlock

$$P_1 \leftrightarrow P_2$$

Processes are blocked by a circular dependency.

- Waiting relationship is cyclic.
- Affected processes cannot progress.
- Resources may remain held indefinitely.

Starvation

P_1 waits

Other processes continue receiving service.

- No deadlock cycle is required.
- Other processes can continue.
- One process may be repeatedly bypassed.

Important Distinction: System Progress

Deadlock

Affected processes: no useful progress

The processes involved in the deadlock remain blocked.

Starvation

System: progress possible; victim process: no progress

Other processes can continue to execute while one process waits indefinitely.

Exam clue

If the question says “a process is repeatedly denied CPU/resource access while other processes continue,” think **starvation**.

Causes of Starvation

Starvation can result from unfair scheduling or resource allocation.

- **Priority scheduling:** continuous arrival of higher-priority processes.
- **SJF/SRTF:** repeated arrival of short jobs can delay a long job.
- **Unfair resource allocation:** some processes repeatedly win resource selection.
- **Repeated victim selection:** the same process may repeatedly be chosen during deadlock recovery.
- **Unfair synchronization:** a waiting thread may repeatedly lose access to a lock/resource.

Key idea

Starvation is primarily a **fairness** problem.

Deadlock Recovery Can Cause Starvation

Deadlock recovery may introduce starvation.

Deadlock $\rightarrow$ Select victim P1 $\rightarrow$ Rollback P1



Deadlock occurs again $\rightarrow$ Select P1 again $\rightarrow \dots$

Problem

If P1 is repeatedly selected as the recovery victim, P1 may lose progress indefinitely.

Mitigation

Include the number of previous rollbacks/preemptions in the victim-selection policy.

Preventing Starvation: Aging

Aging

Aging gradually increases the priority of a process that has waited for a long time.

Long waiting time $\implies$ Higher effective priority

Example

P1 initially has low priority. As its waiting time increases, its priority is gradually improved until it receives CPU service.

- Improves fairness.
- Reduces indefinite waiting.
- Commonly discussed with priority scheduling.

Other Ways to Reduce Starvation

- **Aging**: increase priority as waiting time increases.
- **FIFO queues**: serve requests in arrival order where appropriate.
- **Bounded waiting**: place an upper bound on how long a process can wait.
- **Fair locks**: use queue-based or fair synchronization mechanisms.
- **Fair victim selection**: avoid repeatedly choosing the same process during recovery.

Design goal

Efficiency + Fairness + Bounded waiting

Comparison: Deadlock vs. Starvation

Question	Deadlock	Starvation
Who is affected?	A set of mutually waiting processes	Potentially one process
Why does it happen?	Circular resource dependency	Unfair service/allocation
Do other processes progress?	Processes in the deadlock do not	Yes, other processes may progress
Does a cycle have to exist?	For single-instance resource deadlock, yes	No
Main concern	Resource dependency	Fairness
Typical solution	Prevention, avoidance, detection/recovery	Aging and fair allocation

Exam-Oriented Questions

- 1 Differentiate deadlock and starvation with one OS example each.
- 2 Can starvation occur without deadlock? Explain.
- 3 Can a deadlock lead to starvation? Explain with a recovery scenario.
- 4 Why can priority scheduling cause starvation?
- 5 How does aging reduce starvation?
- 6 A process remains ready for a very long time while other processes continue executing. Is this necessarily deadlock? Justify.
- 7 Compare deadlock, starvation, and livelock.

Summary

- **Deadlock**: processes are blocked by a cyclic dependency on resources.
- **Starvation**: a process waits indefinitely because other processes repeatedly receive service first.
- Deadlock can stop progress for the affected set; starvation can occur while the rest of the system continues.
- Starvation is strongly associated with fairness in scheduling and resource allocation.
- **Aging** is a common technique for reducing starvation.
- Deadlock recovery policies must also consider starvation.

Deadlock = Cyclic waiting

Starvation = Indefinite waiting

Key Takeaway

A simple way to remember

Deadlock: “I cannot proceed because you are waiting for me.”

Starvation: “I cannot proceed because others keep going before me.”

Next topic: Starvation and Livelock